

ROS2 TurtleSim 迷宫探索项目报告

AI Robot Programming Week 14

学生信息

- 姓名：杨奕峰
 - 项目类型：个人项目
 - 项目方向：小乌龟迷宫探索
 - 开发平台：ROS2 Humble + Docker + TurtleSim
-

1. 项目概述

本项目基于 ROS2 和 TurtleSim 仿真平台，实现了小乌龟机器人在迷宫环境中的运动控制与自动探索功能。通过 ROS2 的 Topic 通信机制，小乌龟能够接收运动指令并完成前进、转向等动作，同时结合简单的自动探索策略实现迷宫导航。

本项目的目标如下：

1. 学习 ROS2 基本开发流程；
2. 理解 Publisher 与 Subscriber 通信机制；
3. 实现小乌龟运动控制；
4. 构建迷宫环境；
5. 完成自动探索实验。

选择该项目的理由是 TurtleSim 能够直观展示机器人控制过程，有助于理解机器人导航与路径规划的基本思想。

2. 系统链路设计

本项目采用 ROS2 Topic 通信方式实现控制。

系统整体结构如下：

用户控制端

↓

ROS2 Publisher

↓

/turtle1/cmd_vel

↓

TurtleSim

↓

/turtle1/pose

↓

自动探索程序

其中：

- Publisher 负责发送速度控制指令；
- TurtleSim 根据指令控制小乌龟运动；
- Pose Topic 实时反馈位置信息；
- 自动探索模块根据位置信息决定下一步动作。

主要通信 Topic：

Topic 名称	功能
/turtle1/ cmd_vel	发送速度控制命令
/turtle1/pose	获取当前位置
/rosout	系统日志输出

通过 ROS2 的发布与订阅机制，实现了控制指令与位置反馈之间的数据交互。

3. 迷宫设计

为了验证导航效果，本项目设计了简单迷宫环境。

迷宫包含：

- 起点 (Start)
- 若干障碍物
- 通行区域
- 终点 (Goal)

其中：

- S 为起点；
- G 为终点；
- 为障碍区域。

通过调整障碍物位置和通道宽度，提高探索难度，并验证导航算法的有效性。

4. 代码改动说明

项目主要编写了控制与探索模块。

(1) 运动控制模块

负责发布速度指令控制小乌龟运动。

核心功能：

- 前进
- 停止
- 左转
- 右转

关键代码逻辑：

```
msg.linear.x = 2.0
```

```
msg.angular.z = 0.0
```

```
publisher.publish(msg)
```

该模块通过修改线速度与角速度实现运动控制。

(2) 位置获取模块

通过订阅 Pose Topic 获取实时位置。

主要获取：

- x 坐标
- y 坐标
- 朝向角度

核心逻辑：

```
def pose_callback(msg):
```

```
    self.x = msg.x
```

```
    self.y = msg.y
```

(3) 自动探索模块

根据当前位置判断下一步动作。

主要逻辑：

```
if obstacle_detected:
```

```
    turn_left()
```

```
else:
```

```
    move_forward()
```

当检测到障碍时自动转向，否则继续前进。

5. 自动探索策略

本项目采用简单墙壁跟随 (Wall Following) 策略。

流程如下：

开始

↓

前进

↓

检测障碍

↓

有障碍？

├ 是 → 转向

└ 否 → 继续前进

↓

到达终点

↓

结束

该方法实现简单，适合小型迷宫环境。

实验过程中，小乌龟能够根据障碍物分布不断调整方向，最终到达目标区域。

6.修改内容

这次为了实现小乌龟的“一键自动寻路”功能，我们对整个项目进行了一次完整的软硬件控制闭环重构，主要修改了 index.html (前端网页) 和 turtlesim_web_bridge.py (后端桥接) 两个文件。

以下是为你整理的修改内容总结：

一、前端页面 (index.html) 修改总结

前端的核心任务是提供控制开关、防止控制冲突、并实时反馈小乌龟的寻路状态。

1. 新增“自动寻路”按钮 (UI 层)：

- 在控制面板的“重置到起点”按钮左侧，加入了一个 `<button id="toggleAuto">开启自动寻路</button>` 交互控件。

2. 状态驱动的动态样式 (CSS 层)：

- 增加了 `.success` 绿色背景样式。当小乌龟处于自动寻路时，按钮文字切换为“关闭自动寻路”并变为**绿色**；正常手动模式下为**蓝色**。

3. 增加 WebSocket 信号绑定 (JavaScript 逻辑层)：

- 绑定了按钮的点击事件，每次点击会计算出与当前相反的布尔值 (`!isAutoMode`)，并通过 WebSocket 向后端发送 { type: "toggle_auto", auto: true/false } 协议数据。

4. 互斥安全锁机制 (防止操作冲突)：

- 给手动的四个方向按键加上了 `.dir-btn` 类名。
- 核心逻辑**：当收到后端传来的 `state.rule.auto == true` 时，利用 JS 强制禁用 (**disabled**) 四个方向键，防止学生在自动寻路时误触方向键，导致手动速度与算法速度叠加产生畸变；寻路关闭或结束后，自动恢复方向键。

二、后端桥接 (turtlesim_web_bridge.py) 修改总结

后端的核心任务是打通 A* 算法控制链、做好手动/自动平滑切换、以及边界安全保护。

1. 完善 toggle_auto 消息处理 (平滑切换逻辑)：

- 开启自动时**：不仅将 `bridge.auto` 设为 `True`，还主动将 `bridge.current_linear` 和 `angular` 清零，并清除上一轮残余的 A* 路径点 (`waypoints = None`)，强制触发 Planner 重新规划。

- **中途关闭自动时**：立刻调用 `bridge.stop()`。这非常关键，因为如果不调用，小乌龟会带着自动寻路最后一帧的残余线速度无限向前冲直到撞墙。现在点击关闭，它会瞬间安全刹车。

2. 速度覆盖与控制权交接 (`compute_safe_motion`)：

- 在定时器的每帧物理计算中，加入了条件分支：`if self.auto and not self.goal_reached:`。
- 一旦开启自动，它会屏蔽前端传来的手动速度，改由 `self.explorer.decide(self.get_state())` 实时计算出 A* 算法的推荐线速度和角速度，实现控制权由“人”到“AI”的交接。

3. 联动终止机制（安全保护）：

- **到达终点自动关闭**：在 `on_pose` 监听到小乌龟到达目标点时，将 `self.auto` 改为 `False`，前端按钮会瞬间同步变回蓝色，交回手动控制权。
- **触发撞墙/障碍物保护自动关闭**：在安全策略触发 `boundary`（边界）或 `obstacle`（障碍物）拦截时，同样强制将 `self.auto = False` 强行终止寻路，防止 A* 算法在局部死区不断发出冲撞指令。

三、算法常量 (`explorer.py`) 建议微调（针对原地转圈问题）

为了让小乌龟在网页端点击“自动寻路”后能够更丝滑地起步，而不是由于角度控制震荡卡在原地打转，建议对算法参数进行微调：

- 将转向增益 `K_ANG` 从 3.0 降低到 1.5（防止转向过度、左右横跳）。
- 将先转后走的容忍角度 `TURN_FIRST` 从 0.6 放大到 1.0（约 57°，让它不用对得极准就能提早一边往前走一边修正车头）。

总结

修改后，项目实现了一个非常标准的“前端触发 - 后端切换 - 算法接管 - 状态同步反馈 - 自动触发安全保护/终点停机 - 释放手动控制权”的完整交互闭环。

7. 项目成果

本项目成功实现：

- ✅ ROS2 节点开发
- ✅ Publisher/Subscriber 通信

✅ TurtleSim 控制

✅ 迷宫环境搭建

✅ 自动探索功能

✅ 仿真测试验证

最终实验中，小乌龟能够按照预设策略完成迷宫探索并成功到达终点。

(此处插入 TurtleSim 运行截图)

8. 总结

通过本次项目实践，我掌握了 ROS2 的基本开发流程，理解了 Topic 通信机制以及机器人控制方法。同时学习了 TurtleSim 仿真环境的使用方式，并初步接触了机器人导航与自动探索算法。

项目成功完成了预期目标，实现了小乌龟在迷宫环境中的运动控制和自动探索功能，为后续学习移动机器人导航、路径规划以及 SLAM 等高级技术奠定了基础。